Autor:
Manuel Eugenio Morocho Cayamcela, PhD

Prueba práctica:

Clasificación de Pinguinos de la Isla Palmer con un perceptrón multicapa (MLP)

Objetivo y alcance del trabajo:

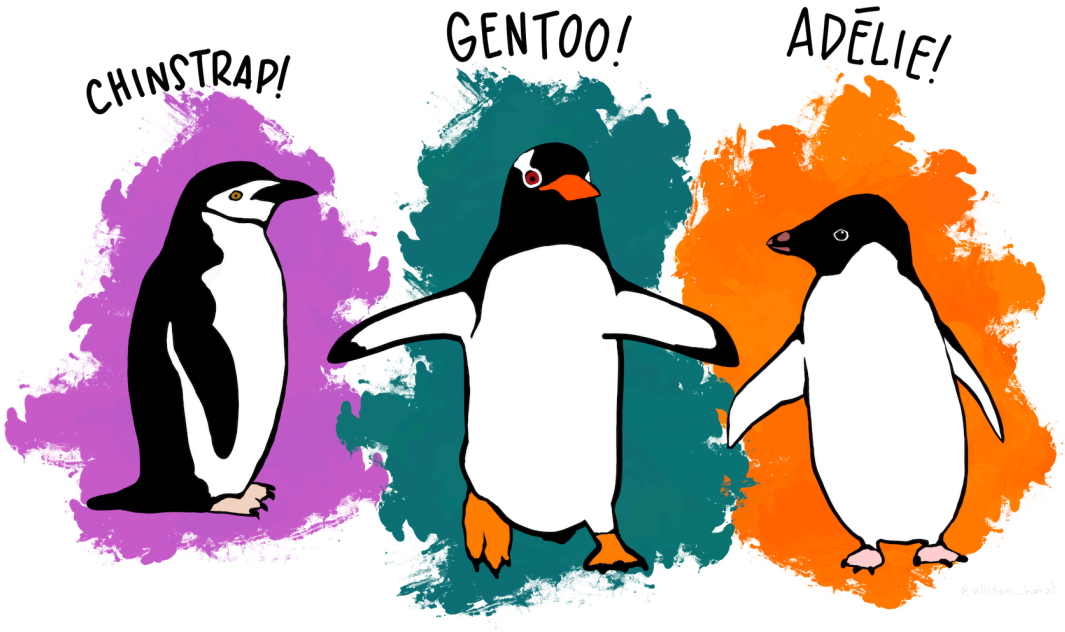
Objetivo:

El objetivo de este Jupyter Notebook es desarrollar y evaluar un modelo de clasificación utilizando un Perceptrón Multicapa (MLP). El modelo se entrenará utilizando un conjunto de datos de entrenamiento y luego se utilizará para hacer predicciones en un conjunto de datos de prueba.

Alcance:

Este Jupyter Notebook cubrirá los siguientes aspectos:

1. Importación de las bibliotecas necesarias.
2. Carga y exploración inicial de los datos.
3. Preprocesamiento básico de los datos.
4. División de los datos en conjuntos de entrenamiento y prueba.
5. Creación de un MLPClassifier.
6. Entrenamiento del MLPClassifier con el conjunto de entrenamiento.
7. Uso del MLPClassifier entrenado para hacer predicciones en el conjunto de prueba.
8. Evaluación del rendimiento del modelo.



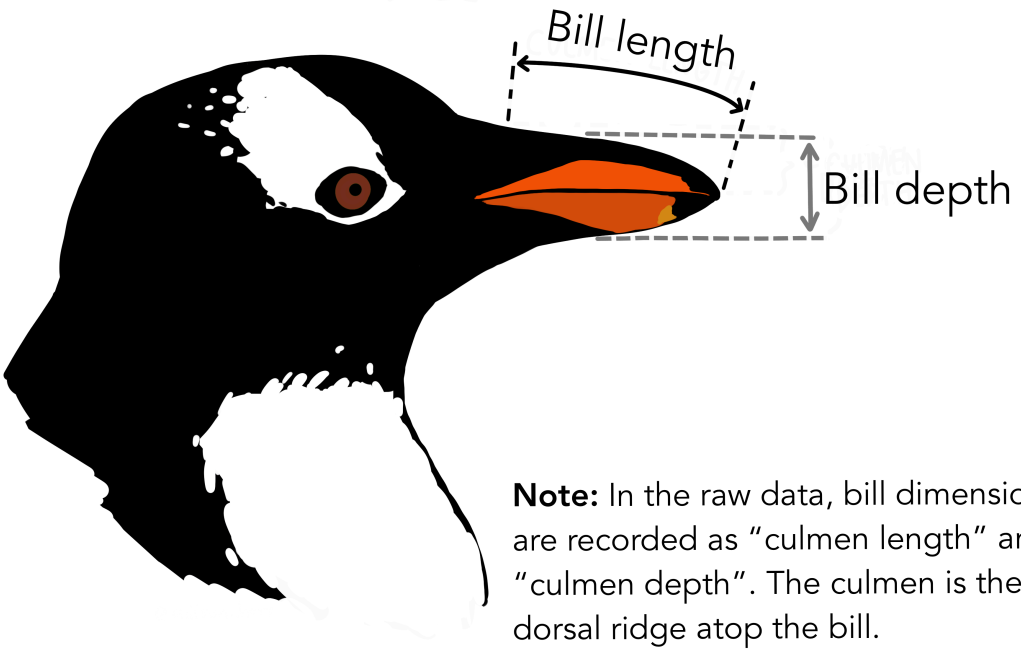
El conjunto de datos de Palmer Penguins fue introducido en 2020 por Allison Horst, Alison Hill y Kristen Gorman. Este conjunto de datos es a menudo utilizado para tareas de clasificación en aprendizaje automático.

El conjunto de datos contiene 344 pingüinos observados en tres diferentes islas del archipiélago de Palmer, en la Antártida. Los datos fueron recogidos por el proyecto Palmer Long Term Ecological Research y la Fundación Nacional para la Ciencia de los Estados Unidos.

Cada pingüino se clasifica en una de las tres especies: Adelie, Chinstrap y Gentoo. Para cada pingüino, el conjunto de datos incluye siete características:

1. Especie: Adelie, Chinstrap, Gentoo.
2. Isla: Torgersen, Biscoe, Dream.
3. Longitud del pico (mm).
4. Profundidad del pico (mm).
5. Longitud de la aleta (mm).
6. Masa corporal (g).
7. Sexo: masculino, femenino.

El objetivo principal al trabajar con este conjunto de datos es a menudo construir un modelo de clasificación que pueda predecir la especie de un pingüino basándose en las características que se tienen en la base de datos.



Note: In the raw data, bill dimensions are recorded as "culmen length" and "culmen depth". The culmen is the dorsal ridge atop the bill.

Fase I: Carga y procesamiento de la base de datos

In [8]:
```
# Importamos las librerías necesarias
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

In [9]:
```
# Cargamos la base de datos 'palmer_penguins.csv' en un DataFrame de Pandas de nomb
pinguinos = pd.read_csv(r"C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\FUNDAMEN

# Mostremos las primeras 5 filas de la base de datos
pinguinos.head()
```

Out[9]:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mas
0	Adelie	Torgersen	39.1	18.7	181.0	37
1	Adelie	Torgersen	39.5	17.4	186.0	38
2	Adelie	Torgersen	40.3	18.0	195.0	32
3	Adelie	Torgersen	NaN	NaN	NaN	N
4	Adelie	Torgersen	36.7	19.3	193.0	34

In [10]:
```
# Usamos el método 'isnull()' para identificar los valores nulos y el método 'sum()
pinguinos.isnull().sum()
```

Out[10]:
```
species              0
island               0
culmen_length_mm     2
culmen_depth_mm      2
flipper_length_mm    2
body_mass_g          2
sex                 10
dtype: int64
```

In [11]:
```
# Usamos el método 'dropna()' para eliminar las filas con valores nulos del DataFra
pinguinos = pinguinos.dropna()

# Usamos el método 'head()' para mostrar las primeras 5 filas de la base de datos
pinguinos.head()
```

Out[11]:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mas
0	Adelie	Torgersen	39.1	18.7	181.0	37
1	Adelie	Torgersen	39.5	17.4	186.0	38
2	Adelie	Torgersen	40.3	18.0	195.0	32
4	Adelie	Torgersen	36.7	19.3	193.0	34
5	Adelie	Torgersen	39.3	20.6	190.0	36

In [12]:
```
# Usamos el método 'unique()' para encontrar los valores únicos en la columna 'sex'
pinguinos['sex'].unique()
```

Out[12]:
```
<StringArray>
['MALE', 'FEMALE', '.']
Length: 3, dtype: str
```

In [13]:
```
# Eliminamos las filas con el valor '.' en la columna 'sex' usando el método de fil
pinguinos = pinguinos[pinguinos['sex'] != '.']
```

Fase II: Preparamos los datos para ser usados en un perceptrón multicapa (MLP)

In [14]:
```
# Aislamos las variables predictoras que son todas menos la variable objetivo
# Usamos el método 'drop()' para eliminar la columna 'species' del DataFrame 'pingu
X = pinguinos.drop('species', axis=1)

# Convertimos las variables categóricas en variables numéricas mediante el método '
# Usamos el método 'apply()' para aplicar la función 'LabelEncoder().fit_transform'
X = X.apply(LabelEncoder().fit_transform)

# Mostremos las primeras 5 filas de la base de datos
X.head()
```

Out[14]:

	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	2	41	56	5	30	1
1	2	44	43	10	32	0
2	2	50	49	19	11	0
4	2	21	62	17	18	0
5	2	43	73	14	26	1

In [15]:
```
# Aislamos la variable objetivo 'species' en la variable 'y'
y = pinguinos['species']

# Mostramos las primeras 5 filas de la base de datos
y.head()
```

Out[15]:
```
0    Adelie
1    Adelie
2    Adelie
4    Adelie
5    Adelie
Name: species, dtype: str
```

In [16]:
```
# Dividimos los datos 'X' y 'y' en entrenamiento y prueba con el método 'train_test
# Usamos el 0.2 (20%) de los datos para prueba
# Guardamos los datos de entrenamiento en las variables 'X_train' y 'y_train'
# Guardamos los datos de prueba en las variables 'X_test' y 'y_test'
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Fase III: Clasificación de pinguinos con un perceptrón multicapa (MLP)

In [17]:
```
# Creamos un clasificador de tipo MLPClassifier con 2 capas ocultas de 100 neuronas
# Usamos la función de activación 'relu' y el optimizador 'adam'
mlp = MLPClassifier(hidden_layer_sizes=(100, 100), activation='relu', solver='adam'

# Usamos el método 'fit()' para entrenar el clasificador
# Entrenamos el clasificador con los datos de entrenamiento 'X_train' y 'y_train'
mlp.fit(X_train, y_train)

# Predecimos las etiquetas de los datos de prueba 'X_test' usando el método 'predic
y_pred = mlp.predict(X_test)
y_pred
```

Out[17]:
```
array(['Chinstrap', 'Adelie', 'Adelie', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Gentoo', 'Chinstrap', 'Adelie', 'Gentoo', 'Gentoo',
       'Gentoo', 'Gentoo', 'Gentoo', 'Gentoo', 'Chinstrap', 'Adelie',
       'Gentoo', 'Gentoo', 'Gentoo', 'Chinstrap', 'Gentoo', 'Gentoo',
       'Gentoo', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo',
       'Adelie', 'Gentoo', 'Chinstrap', 'Chinstrap', 'Adelie', 'Adelie',
       'Gentoo', 'Adelie', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie',
       'Adelie', 'Chinstrap', 'Gentoo', 'Gentoo', 'Gentoo', 'Chinstrap',
       'Gentoo', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo', 'Adelie',
       'Adelie', 'Adelie', 'Adelie', 'Chinstrap', 'Chinstrap', 'Adelie',
       'Adelie', 'Adelie', 'Chinstrap', 'Gentoo', 'Gentoo', 'Chinstrap',
       'Gentoo'], dtype='<U9')
```

In [18]:
```
# Calculamos la precisión del modelo de clasificación 'mlp' con el método 'accuracy
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'accuracy_scor
# Guardamos el resultado en la variable 'accuracy'
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

Out[18]:
```
0.9850746268656716
```

In [19]:
```
# Mostramos la matriz de confusión del modelo de clasificación 'mlp' con el método
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'confusion_mat
confusion_matrix(y_test, y_pred)
```

Out[19]:
```
array([[26,  1,  0],
       [ 0, 13,  0],
       [ 0,  0, 27]])
```

In [20]:
```
# Imprimimos el reporte de clasificación del modelo de clasificación 'mlp' con el m
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'classificatio
# Imprimimos el reporte de clasificación en la consola con la función 'print'
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

      Adelie       1.00      0.96      0.98        27
   Chinstrap       0.93      1.00      0.96        13
      Gentoo       1.00      1.00      1.00        27

    accuracy                           0.99        67
   macro avg       0.98      0.99      0.98        67
weighted avg       0.99      0.99      0.99        67
```

In [23]:
```
# Hacemos una prueba de predicción de especie con valores de un nuevo pinguino (no
# Creamos una lista con los valores de las variables predictoras del nuevo pinguino
# Puedes usar estos valores de ejemplo o cambiarlos por los valores aleatorios de u
#   island = 1 (Biscoe)
#   culmen_length_mm = 45
#   culmen_depth_mm = 58
#   flipper_length_mm = 11
#   body_mass_g = 22
#   sex = 0 (FEMALE)
# Guardamos la lista en la variable 'nuevo_pinguino'
nuevo_pinguino = [[1, 45, 58, 11, 22, 0]]
```

```
# Hacemos una predicción de la especie del 'nuevo_pinguino' con el método 'predict'
# Guardamos la predicción en la variable 'nueva_prediccion'
nueva_prediccion = mlp.predict(nuevo_pinguino)
nueva_prediccion
```

```
c:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\FUNDAMENTOS DE INTELIGENCIA ARTIFIC
IAL\.venv\Lib\site-packages\sklearn\utils\validation.py:2830: UserWarning: X does no
t have valid feature names, but MLPClassifier was fitted with feature names
  warnings.warn(
```

Out[23]:
```
array(['Adelie'], dtype='<U9')
```

In [24]:
```
# Guardar los pesos (coeficientes) de las capas del modelo MLP en archivos CSV
for i, coef in enumerate(mlp.coefs_):
    np.savetxt(f"pesos_capa_{i+1}.csv", coef, delimiter=",")

# Guardar los sesgos (bias) de las capas del modelo MLP en archivos CSV
for i, intercept in enumerate(mlp.intercepts_):
    np.savetxt(f"sesgos_capa_{i+1}.csv", intercept, delimiter=",")

# Esto generará archivos pesos_capa_1.csv, pesos_capa_2.csv, etc., que contienen lo
# Y archivos sesgos_capa_1.csv, sesgos_capa_2.csv, etc., que contienen los sesgos d

# Nota 1: Asegúrate de que el modelo 'mlp' ya ha sido entrenado antes de ejecutar e
# Nota 2: MLPClassifier usa ReLU en las capas ocultas y sigmoid en la salida (para
# Nota 3: El orden de los logits de salida corresponde al orden alfabético de las c
```

Entregar el archivo `.pdf` generado de este notebook:

1. Usa los pesos generados por el `MLPClassifier` para replicar la inferencia del modelo con el nuevo pinguino en Excel.

Inferencia en Excel siguiendo la plantilla `MLPClassifier_Excel_inference.xlsx` (archivo `MLPClassifier_Excel_inference_completo.xlsx`), con los pesos de `pesos_capa_k.csv` y sesgos de `sesgos_capa_k.csv` del modelo entrenado arriba.

1) Entrada, pesos de la 1.ª capa oculta, bias, `MMULT` (nuevo pingüino × pesos), suma del bias y ReLU:

New Penguin					
1	45	58	11	22	0

Weights from the 1st hidden layer

0	1	2	3	4	5	6	7	8	9	10	11
-0,05241894	0,22557057	0,05410247	0,03472514	-0,1557393	-0,09816077	-0,13963708	0,18266027	0,01313258	0,04546941	-0,15596123	0,23951956
-0,21546781	0,06265747	-0,03773604	-0,00261989	0,19028889	-0,1347259	-0,01056249	0,13154649	-0,1079234	-0,13149014	-0,04623934	-0,14506884
0,07466632	-0,19052301	-0,09583956	0,17939804	0,05540135	-0,2506364	-0,12102169	0,08822869	-0,21505071	-0,09617713	0,00349222	0,10604489
-0,20587463	0,01348287	0,00252835	0,05415044	0,11238506	0,21086385	0,00056598	-0,07832495	0,16149433	-0,05306407	-0,00524436	-0,18676998
-0,1814139	0,19491686	0,00012264	0,14408535	-0,08063443	0,17266387	-0,01548341	-0,22385132	0,21381405	-0,12536178	-0,03608057	0,228627
0,04200515	0,02806296	-0,039368	0,1372389	0,09505217	-0,1769737	0,12630772	0,16336524	0,23212684	0,05180352	0,01614208	-0,02528507

Bias from the 1st hidden layer

0
-0,15035127
-0,10128246
-0,15368841
-0,20713167
-0,17412457
-0,03417371
-0,13973551
-0,05594655
0,02264405

New Penguin x Weights from the 1st hidden layer

-11,6735505	-3,56869552	-7,1722041	14,0874487	11,0828174	-14,5796298	-7,96861625	5,4332132	-10,8360147	-14,7915244

(New Penguin x Weights from the 1st hidden layer) + Bias from the 1st hidden layer

-11,8239017	-3,66997797	-7,32589251	13,8803171	10,9086929	-14,6138035	-8,10835176	5,37726665	-10,8133706	-14,7009287

ReLU from the 1st hidden layer

0	0	0	13,8803171	10,9086929	0	0	5,37726665	0	0

2) 2.ª capa oculta: pesos, `MMULT(h1; W2)` , + bias y ReLU:

Weights from the 2nd hidden layer

0	1	2	3	4	5	6	7	8	9
0,00101689	-0,09088713	-0,01757016	-0,06698998	-0,08746898	0,10341989	-0,06856161	0,05175704	-0,07663605	0,02114411
0,02660667	-0,06194175	0,038424	0,04899479	-0,0275202	0,07777495	0,15712069	0,13491643	-0,03014565	-0,14430529
-0,04709707	-0,09844327	-0,05407469	0,00515511	-0,00466276	0,07467805	0,0537744	-0,01658671	-0,03425457	-0,01013153
-0,05308568	0,00154119	0,11994816	0,0707278	0,09882035	0,04711054	0,05655521	0,1086254	-0,07831065	-0,00691417
-0,08910331	0,09713238	0,12778419	0,14242693	0,16924462	-0,11285063	-0,04105661	0,07696519	0,07163827	-0,12327981
0,0385362	-0,09902832	-0,15034812	-0,04564641	-0,00972087	0,08137723	0,08165275	-0,0313484	-0,09394442	0,15848502
-0,09692578	0,08734724	-0,09084327	0,00150384	0,02700888	0,06850494	0,02812849	0,04960643	-0,01703065	0,05307448
-0,05796379	-0,05780665	-0,16852465	0,07427526	0,05903904	5,0257E-05	0,11660561	0,05064439	0,03584815	0,13131729
0,00044671	-0,00050768	-0,14672228	-0,03698594	-0,04396982	-0,04769418	0,04871491	-0,00135831	0,01929685	0,07716636
-0,05129539	-0,02015877	-0,04229439	-0,01419804	-0,0856879	0,00044596	-0,08622058	0,0492465	-0,04052435	0,00142602
0,09614903	-0,0035449	-0,02252118	0,000101	-0,00291437	-0,07515541	0,0134714	-0,04499507	0,01002164	0,01512292
0,0933637	0,03352668	-0,06403704	-0,08366276	-0,06577153	0,08189036	0,04247376	-0,05306896	-0,09985685	-0,12309046
0,0024032	-0,10487819	0,08352722	-0,16836449	0,08111989	-0,10462015	0,15170968	-0,0581066	-0,05464007	-0,12519708
-0,0337127	-0,03725244	0,14341876	-0,08481608	-0,08053112	0,08887503	-0,01779391	0,08336512	-0,15435352	-0,01155204
0,08115719	0,00103919	-0,12563167	0,05998375	0,09619112	-0,07001311	0,11540866	0,1282524	-0,07145551	0,16276317
0,09944586	-0,07370642	-0,02329702	-0,15533377	0,08421057	0,14688395	-0,07385072	0,11159728	-0,07674989	0,13579953
0,04983179	0,00398593	0,00603614	-0,06903554	0,16120452	-0,02689398	-0,13606458	0,02812619	-0,13781851	-0,05597975
-0,01681368	-0,08501582	-0,04129213	-0,07666593	0,02025752	0,02816989	-0,0586563	-0,04452032	0,17945071	-0,00959668
0.056487	0.04008595	0.01871639	0.15076814	-0.1068786	-0.13580067	0.11626216	-0.03348043	0.12534112	0.13294053

Output from the 1st hidden layer x Weights from the 2nd hidden layer

-5,87209409	-1,48722466	-0,2089684	1,23200438	-1,63026419	-2,31499835	5,05779548	-14,0653379	4,48072051	-3,01639829	-0,92350517	-2,51020108

en layer

0
-0,10933805
-0,01971908
-0,12446793
0,15755396
-0,14752364

(Output of the 1st hidden layer x Weights from the 2nd output layer) + Bias from the 2st hidden layer

-5,98143214	-1,50694373	-0,33343633	1,38955834	-1,77778783	-2,29318334	5,09126762	-14,0648301	4,54486559	-3,15425167

ReLU from the 2nd hidden layer

0	0	0	1,38955834	0	0	5,09126762	0	4,54486559	0

3) Capa de salida: pesos, `MMULT(h2; W3)` , + bias (logits), softmax y especie predicha:

Weights from the 3rd layer (output layer)

0	1	2
-0,02662547	0,09738138	-0,13857761
0,14259235	0,16207359	-0,12907014
-0,09350323	-0,06039141	0,12730541
0,05469429	0,06542918	0,1218635
-0,18414987	0,16270104	0,09980429
0,04414345	0,18086865	0,10156326
-0,13524419	-0,03434924	-0,0103798
-0,22269516	0,2073719	-0,16711785
0,09510791	0,00518643	-0,22397806
0,19810504	0,1800887	0,01252109
-0,11778246	-0,19623089	-0,07645521
-0,11139611	-0,0036838	0,22872526
0,0078309	0,11976424	0,02727131
-0,00111525	0,10468496	-0,00141919
0,11063642	-0,04666112	-0,11172754
0,18464148	0,03002276	0,17550216
0,21854446	-0,22362579	-0,00516859
-0,0531979	0,05999972	0,13697589
-0.09287751	0.01436582	0.08902617

Output from the 2nd hidden layer x Weights from the output layer

17,3804472	2,06861837	-4,90002368

Bias from the output layer

0
-0,09948678
0,17579793
0,1207207

(Output layer x Weights from the output layer) + Bias from the output layer

17,2809604	2,24441629	-4,77930298

Softmax from the output layer

0,9999997	2,9493E-07	2,6263E-10

$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Adelie	Chinstrap	Gentoo

Predicted species	**Adelie**

4) Detalle de las fórmulas usadas en cada paso:

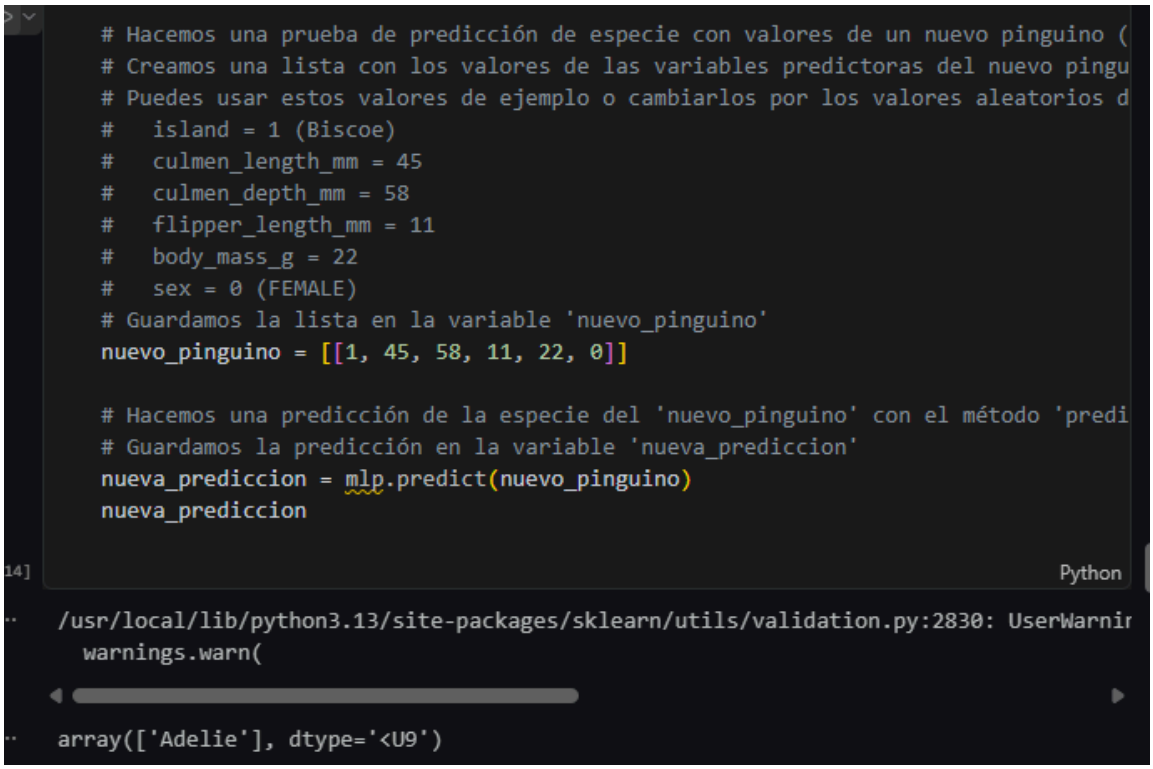
Paso	Celdas	Fórmula usada
1. Entrada	A3:F3	Nuevo pingüino (valores codificados): 1, 45, 58, 11, 22, 0
2. Capa oculta 1: x · W1	C17:CX17	=MMULT(A3:F3,A7:CV12)
3. + bias 1	C20:CX20	=C17:CX17+TRANSPOSE(A16:A115)
4. ReLU 1	C23:CX23	=MAX(0,C20) (copiada a la derecha)
5. Capa oculta 2: h1 · W2	C129:CX129	=MMULT(C23:CX23,C27:CX126)
6. + bias 2	E133:CZ133	=C129:CX129+TRANSPOSE(C133:C232)
7. ReLU 2	E136:CZ136	=MAX(0,E133) (copiada a la derecha)
8. Capa de salida: h2 · W3	I141:K141	=MMULT(E136:CZ136,E140:G239)
9. + bias salida (logits)	I150:K150	=I141:K141+TRANSPOSE(I145:I147)
10. Softmax	I153:K153	=EXP(I150)/SUM(EXP($I$150:$K$150)) (ídem J y K)
11. Especie predicha	K157	=INDEX(I155:K155,MATCH(MAX(I153:K153),I153:K153,0))

2. Responde a la siguiente pregunta: ¿Cuál es la especie predicha para el nuevo pinguino usando el método `.predict`?

Respuesta: con `mlp.predict(nuevo_pinguino)` la especie predicha es **Adelie**.

Probabilidades de `predict_proba`: Adelie = 0.99999970, Chinstrap = 2.95×10^{-7}, Gentoo = 2.63×10^{-10}.

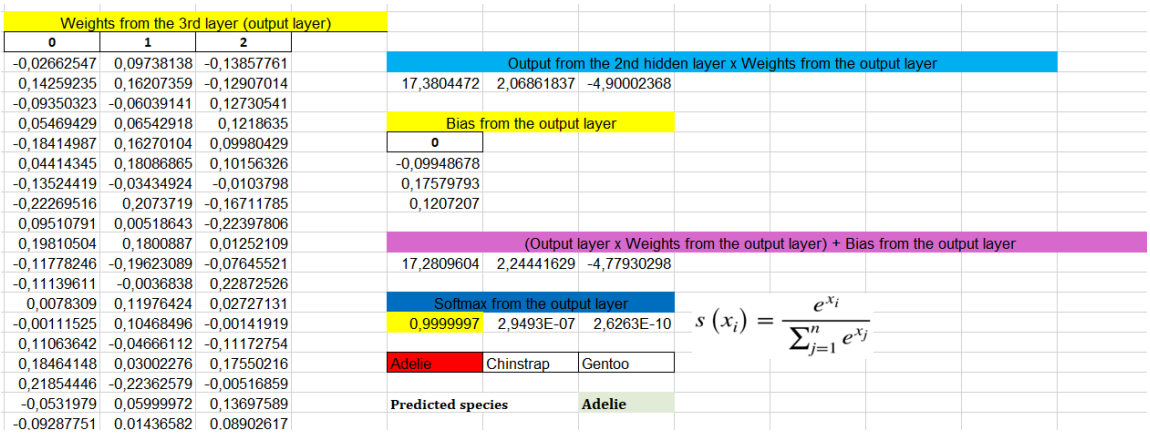
Nota: los valores del ejemplo (45, 58, 11, 22) son **índices de `LabelEncoder`**, no medidas en mm/g, y `island = 1` corresponde en realidad a *Dream* (Biscoe = 0, Dream = 1, Torgersen = 2). Se usaron tal cual los valores indicados en el enunciado.



3. Responde a la siguiente pregunta: ¿Cuál es la especie predicha para el nuevo pinguino usando Excel?

Respuesta: con Excel (`MMULT` + ReLU + softmax) la especie predicha es **Adelie**, idéntica a la de `.predict` .

Logits en Excel: Adelie = 17.2810, Chinstrap = 2.2444, Gentoo = −4.7793 → softmax: 0.99999970, 2.95×10^{-7}, 2.63×10^{-10}, que coinciden con `predict_proba` de scikit-learn (el modelo usa softmax en la salida al haber 3 clases).



Weights from the 3rd layer (output layer)		
0	1	2
-0,02662547	0,09738138	-0,13857761
0,14259235	0,16207359	-0,12907014
-0,09350323	-0,06039141	0,12730541
0,05469429	0,06542918	0,1218635
-0,18414987	0,16270104	0,09980429
0,04414345	0,18086865	0,10156326
-0,13524419	-0,03434924	-0,0103798
-0,22269516	0,2073719	-0,16711785
0,09510791	0,00518643	-0,22397806
0,19810504	0,1800887	0,01252109
-0,11778246	-0,19623089	-0,07645521
-0,11139611	-0,0036838	0,22872526
0,0078309	0,11976424	0,02727131
-0,00111525	0,10468496	-0,00141919
0,11063642	-0,04666112	-0,11172754
0,18464148	0,03002276	0,17550216
0,21854446	-0,22362579	-0,00516859
-0,0531979	0,05999972	0,13697589
-0,09287751	0,01436582	0,08902617

Output from the 2nd hidden layer x Weights from the output layer		
17,3804472	2,06861837	-4,90002368

Bias from the output layer
0
-0,09948678
0,17579793
0,1207207

(Output layer x Weights from the output layer) + Bias from the output layer		
17,2809604	2,24441629	-4,77930298

Softmax from the output layer		
0,9999997	2,9493E-07	2,6263E-10

$$s\left(x_i\right)=\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Adelie	Chinstrap	Gentoo

Predicted species	Adelie

Useful Resources:

- https://support.microsoft.com/es-es/office/función-mmult-40593ed7-a3cd-4b6b-b9a3-e4ad3c7245eb

Prepared by:
Manuel Eugenio Morocho-Cayamcela